Ocean Engineering 312 (2024) 119288



Contents lists available at ScienceDirect

Ocean Engineering

journal homepage: www.elsevier.com/locate/oceaneng



Research paper

Reliable LiDAR-based ship detection and tracking for Autonomous Surface Vehicles in busy maritime environments

Yongchang Xie [a], Cassandra Nanlal [b], Yuanchang Liu [a,*]

[a] Department of Mechanical Engineering, University College London, Roberts Engineering Building, Torrington Place, London, WC1E 7JE, UK
[b] Department of Civil, Environmental and Geomatic Engineering, University College London, Chadwick Building, Gower Street, London, WC1E 6BT, UK

ARTICLE INFO

Dataset link: https://figshare.com/articles/dataset/Maritime_LiDAR-based_Ship_Detection_Datasets/26144536

Keywords:
LiDAR-based perception
Ship detection
Deep learning
Object tracking
Autonomous Surface Vehicle

ABSTRACT

Environmental perception is a crucial requirement of Autonomous Surface Vehicles (ASVs) if required to perform tasks safely in a dynamically complex operational environment. Most existing methods for ship detection rely on camera-based methods, which are sensitive to environmental conditions and cannot directly provide spatial location information related to detected targets. To overcome this limitation, we propose a LiDAR-based ship detection and tracking framework that can be applied to busy maritime environments. The proposed framework consists of two functional modules: a ship detection and multi-object tracking. For ship detection, a modularised network structure was adapted, allowing for ease of switching between different types of detection network to prioritise either detection accuracy, detection speed or a compromise of both, depending on the task requirements. A Kalman Filter-based multi-object tracking method is also implemented to compensate for any detections that may have been missed as a result of ship motions or occlusions, relying solely on the detection results. We also collected the first-ever real-world LiDAR dataset for maritime applications across the River Thames and marinas, including a range of ship types, with lengths ranging from 5 m up to 40 m, and different hull types. The datasets are organised in a similar manner to the KITTI datasets, which can be easily applied to the well-developed point cloud detection networks. Remarkably, our methods achieve an overall detection accuracy of 74.1% in the collected datasets. The proposed framework and dataset make LiDAR-based environmental perception feasible for implementation in ASVs and support development in the autonomous maritime navigation field.

1. Introduction

Autonomous Surface Vehicles (ASVs), with the ability to function as intelligent robots in the maritime domain, have attracted significant attention due to their low development and operational costs. They offer enhanced flexibility and autonomy in complex and hazardous environments, proving highly effective in performing repetitive tasks (Liu et al., 2016). A reliable environmental perception capability is essential for ASVs to perform diverse tasks in various environments, as it helps in operational planning and better ensure safe task execution. In recent years, significant progress has been made in environmental perception technology for Unmanned Ground Vehicles (UGVs). However, advancements in the ASV domain have, by comparison, been slower. This difference is primarily due to the unique challenges presented by maritime environments, such as the impact of wind, waves, tidal and ocean currents on ship movement, as well as performance issues with certain sensor technologies caused by sea fog and water reflection (Liu et al., 2016; Cao et al., 2020). Additionally, maritime traffic is often not subject to strict lane restrictions, and water borne objects at sea can vary widely in size, from small buoys to large ocean going tankers.

Environmental perception involves determining the positional aspect and dynamic state of the ASV and relating these to the current situational map of the surrounding environment. This research focuses on object detection and tracking tasks. Most existing methods for ship detection rely on cameras for ranges that are less than a kilometre or use legally required X-band radars to detect large ships that are at distances greater than a kilometre. However, the detection capability of radar drops significantly in the near field (less than 200 m), especially for small ships (Islam et al., 2013). Camera-based methods have more powerful detection capabilities in the near field but are hindered by numerous limitations in practical applications, such as compromised image capture resulting from the random and unsteady light intensity caused by water reflection and image blurring caused by sea fog (Cao et al., 2020). Furthermore, image-based methods cannot provide real world positional information of targets that are detected without being

* Corresponding author.
E-mail addresses: yongchang.xie.22@ucl.ac.uk (Y. Xie), c.nanlal@ucl.ac.uk (C. Nanlal), yuanchang.liu@ucl.ac.uk (Y. Liu).

https://doi.org/10.1016/j.oceaneng.2024.119288
Received 10 July 2024; Received in revised form 3 September 2024; Accepted 15 September 2024
0029-8018/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

augmented by other detection systems, restricting their application in planning systems.

In contrast, LiDAR is a viable alternative sensor for object detection tasks in the near field due to its robustness under a wide range of weather and lighting conditions (Shan et al., 2021). Additionally, LiDAR generates detailed point cloud data that includes precise spatial locations. These capabilities offer LiDAR as a more effective sensor than cameras for detecting floating objects in marine environments. Despite these advantages, research in this field remains limited. A significant challenge is the complexity of processing point cloud data compared to images. Point clouds, being 3D and containing more semantic information than 2D images, require higher computational resources for processing. This demand affects processing time and the capability for real-time navigation and obstacle avoidance, especially in areas where the density of shipping traffic is high. It should be noted that, the detection quality from both cameras and LiDAR is heavily influenced by the motion of the ASV and occlusion, which increase the potential for there to be missed detections and discontinuities in the detection sequence, which can impose safety issues (Shan et al., 2021; Lin et al., 2022).

Deep-learning-based methods can offer reliable solutions for issues that may arise with LiDAR-based detection, and the LiDAR performance with these deep learning methods has been widely validated in UGV applications. However, compared to the vast number of related LiDAR datasets in the UGV field, the available LiDAR datasets for object detection in the maritime field are limited. The datasets proposed by Lin et al. (2022) are the only relevant datasets that could be found online. The Robotic Operating System (ROS) was used to generate a series of simulated datasets, including different types of ships ranging from 4 m to 20 m in length as well as a variety of buoys. However, detection networks trained with these datasets perform poorly in real-world tests in busy maritime environments, such as marinas and busy channels, as shown in Fig. 1. These networks are unable to reliably distinguish between the point clouds of target ships and background noise, which may include point clouds of buildings, bridges, trees, and other structures. For example, bridge piers shaped like ship hulls are often misidentified as ships, as shown in Fig. 1(b). Moreover, real-world ships, such as the Uber Boat running on the River Thames with a length of 38 m, are not detected despite sufficient point cloud data being collected.

Aware of the advantages of LiDAR-based detection over image-based methods, this project aims to develop a framework using LiDAR as the primary sensor for perception tasks, focusing on busy maritime environments as such environments present additional complexity. To address the challenges of processing 3D point cloud data and accurately detecting ships, we employ Convolutional Neural Network (CNN)-based methods. To mitigate the issue of discontinuous detection caused by ASV motion and occlusion, we introduce a multi-object tracking module that combines the Kalman Filter (KF) with the Hungarian matching algorithm to compensate for missed detections. Using this strategy, we have collected the first-ever real-world maritime LiDAR dataset for ship detection. The main contributions of this paper are as follows:

- We propose a flexible LiDAR-based environmental perception framework for ship detection and tracking by adapting modular detection networks to meet various task requirements. For open water areas requiring fast response, we use PointPillar. For crowded marinas needing accurate detection, we use PV-RCNN.
- We have collected the first-ever real-world maritime LiDAR dataset for ship detection to fill the gap in this area and accelerate the development of the maritime autonomous community. This collected dataset includes a total of 9200 frames comprising different busy maritime environment scenarios, including ships at harbour and moving around the marina and the River Thames.
- The proposed environmental perception framework was evaluated using simulated datasets, achieving a 25.3% increase in mean Average Precision (mAP). It was also tested on our real-world dataset. PointPillar networks had the fastest inference time at 8.4 ms per frame, while PV-RCNN achieved the highest detection accuracy with a 74.1% mAP. The multi-object tracking modules effectively compensated for missed detections, enhancing overall performance.

This paper is organised as follows: Section 2 reviews related works, and Section 3 describes the methods used in our approach. In Section 4, we evaluate and discuss the results of our proposed approach. Finally, the conclusions are presented in Section 5.

2. Related work

2.1. Object detection

In maritime perception tasks, many researchers have employed image-based methods, including object detection and segmentation. For object detection CNN architectures, such as the well-known YOLO framework and its variants, are used Zhang et al. (2024) and Jiang et al. (2024). Segmentation methods involve subtracting the background from the image to isolate the foreground as a region of interest (ROI), from which elements can then be identified as target objects. Cao et al. (2020) proposed using image segmentation with CNNs to extract object features to enable automatic ship recognition. Yao et al. (2021) introduced ShorelineNet, which achieved credible performance in coastline segmentation. Hansen et al. (2023) improved the robustness of obstacle detection by capturing sequential information through the innovative integration of Convolutional LSTM with ShorelineNet. However, phenomena such as glare and water reflections, which lead to image blurring, have resulted in false detections of objects when applying the methods mentioned above. To address the limitations of image-based detection methods in providing real-world object positional location detail, image based detection technology is often fused with additional sensors such as LiDAR to obtain more comprehensive spatial information. Zhang et al. (2021) proposed fusing LiDAR and vision system methods to extract targets, using an integrating operation to merge image and point cloud data.

In the limited research on using LiDAR for open water detection applications, most studies have relied on Euclidean or DBSCAN clustering methods (Lin et al., 2020a; Stanislas and Dunbabin, 2019) to cluster target objects. Shan et al. (2021) proposed a segmentation method using a lightweight network, LiSeg, demonstrating that deep learning-based segmentation methods are more competitive than traditional approaches for distinguishing the foreground area of interest from the background noise. Nonetheless, they still rely on Euclidean clustering to cluster three-dimensional point clouds into multiple objects in post-processing. These clustering methods can become unstable, especially in crowded docking scenarios. Because such methods heavily rely on the predefined Euclidean distance for clustering, distinct objects that are close together can easily be incorrectly grouped within the threshold (Lin et al., 2022).

Apart from using clustering methods to detect objects, deep-learning-based methods have proven more effective in processing point cloud data, with enhanced performance of these deep-learning-based methods being evidenced in ground vehicle applications. LiDAR-based deep-learning features three main approaches: the raw point cloud, Bird's Eye View (BEV) map, and the voxel grid method. Raw point cloud methods process raw 3D point clouds directly without converting them to another representation, with networks such as PointNet (Charles et al., 2017) capturing complex geometrical structures by directly operating on 3D points. BEV methods convert 3D point clouds into 2D pseudo-images, typically using pillars, and employ 2D CNNs for processing (Lang et al., 2019). Voxel grid methods divide the

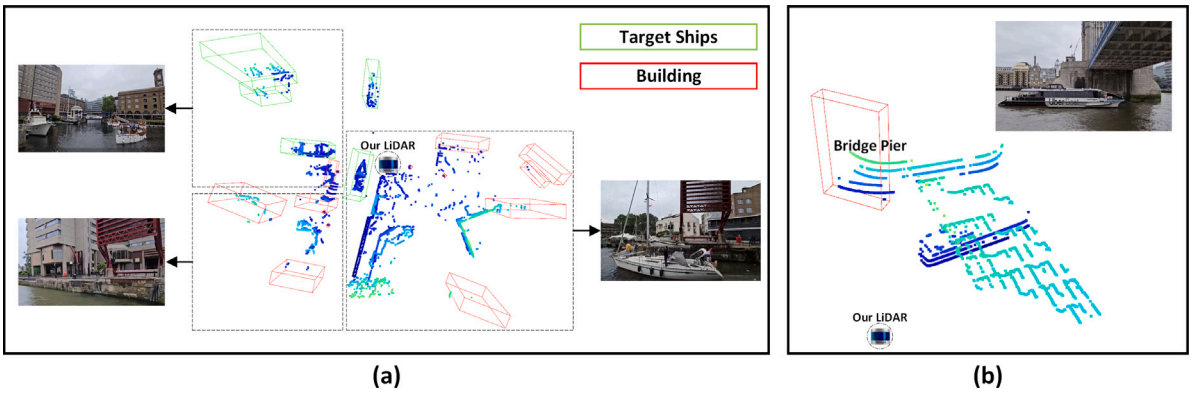


Fig. 1. Real-world detection results for networks trained on simulated datasets. (a) In the busy marina scenario. (b) In the River Thames scenario. Red bounding boxes indicate incorrect detections, while green bounding boxes indicate correct detections.

Table 1
Overview of different point cloud-based detection networks. Detection performance is categorised as ***medium*** (60%–65% mAP), ***high*** (65%–70% mAP), and ***very high*** (above 70% mAP). Inference speed is categorised as ***high*** (less than 20 ms), ***medium*** (20 ms to 50 ms), and ***slow*** (more than 50 ms).

Detection networks	Methods	Stages	Detection performance	Inference speed
PointNet (Charles et al., 2017)	Raw point cloud	1	Medium	High
PointPillar (Lang et al., 2019)	BEV	1	Medium	High
SECOND (Yan et al., 2018)	Voxel	1	High	Medium
PointRCNN (Shi et al., 2019)	Raw point cloud	2	Very High	Slow
PartA2-Net (Shi et al., 2021b)	Voxel	2	Very high	Slow
PV-RCNN (Shi et al., 2021a)	Voxel	2	Very high	Slow

space into voxels and use sparse 3D convolutional layers to process the point cloud (Yan et al., 2018).

Voxel grid methods generally require higher computational resources due to the use of 3D CNNs, whereas BEV methods are less computationally demanding as they use 2D CNNs (Shi et al., 2021a; Pitropov et al., 2022). Raw point cloud methods are often more straightforward in capturing complex geometries and may offer better detection accuracy, though they are not necessarily amongst the better rated methods when it comes to computational resource requirements. The methods mentioned above are all one-stage detection methods. There are also two-stage detection methods that are built upon these stages, where candidate object regions are generated using methods such as Selective Search or Region Proposal Networks (RPN). Two-stage detectors are generally more accurate, as they refine predictions through an additional step, but they are computationally more intensive and slower than one-stage detectors (Shi et al., 2021a,b, 2019). Table 1 presents an overview of different point cloud-based detection networks.

LiDAR-based methods with deep-learning algorithms are selected in this research. This is due to the limitations of image-based detection methods and clustering-based point cloud detection methods. PointPillar, SECOND, and PV-RCNN are implemented in this study since they offer a variety of comparative methods, stages and performance. Comparison studies are conducted to evaluate performance across different networks. The dataset used to train and evaluate the networks is split between simulated data from Lin et al. (2022) and our collected real-world data.

2.2. Datasets for autonomous vehicles

There are many open datasets published for the training of object detection networks. Based on the methods used for object detection, the datasets can generally be separated into image datasets captured by cameras or point cloud datasets captured by LiDAR, stereo cameras, or radar. Table 2 shows some of the most cited relevant datasets. The relevant image datasets used for object detection in autonomous vehicle

Table 2
Most cited perception datasets for autonomous vehicles.

Datasets	Image	Point cloud
Unmanned ground vehicles		
nuScenes (Caesar et al., 2020)	✓	✓
Cityscapes (Cordts et al., 2016)	✓	
BDD100K (Yu et al., 2020)	✓	
Waymo (Hu et al., 2023)	✓	✓
KITTI (Geiger et al., 2013)	✓	✓
SYNTHIA (Ros et al., 2016)	✓	
Apolloscape (Huang et al., 2020)	✓	✓
CADC (Pitropov et al., 2021)	✓	✓
Unmanned aerial vehicles		
UAVDT (Du et al., 2018)	✓	
DroneVehicle (Sun et al., 2022)	✓	
VisDrone (Zhu et al., 2018)	✓	
SeaDronesSee (Varga et al., 2022)	✓	
CARPK (Hsieh et al., 2017)	✓	
FLAME (Shamsoshoara et al., 2021)	✓	
Autonomous surface vehicles		
MaSTr1325 (Bovcon et al., 2019)	✓	
MaSTr1478 (Zust and Kristan, 2022)	✓	
MODS (Bovcon et al., 2022)	✓	
SMD (Prasad et al., 2017)	✓	
Maritime perception (Lin et al., 2022)		✓

applications, including UGVs, Unmanned Aerial Vehicles (UAVs), and ASVs, are quite numerous. There are many well-structured and high-quality point cloud datasets in the UGV field; however, there is a significant gap in relevant point cloud datasets for drones and ASVs.

The work of Lin et al. (2022) is of particular interest to this research. They simulated a series of maritime LiDAR datasets using the ROS to identify multiple targets of interest, including ships, containers and buoys. The simulated datasets were used to train the PointPillar networks which achieved an overall 60.8% mAP score. However, the generalisation of the networks trained with simulated datasets in real-world applications is poor. Besides that, there are no relevant datasets

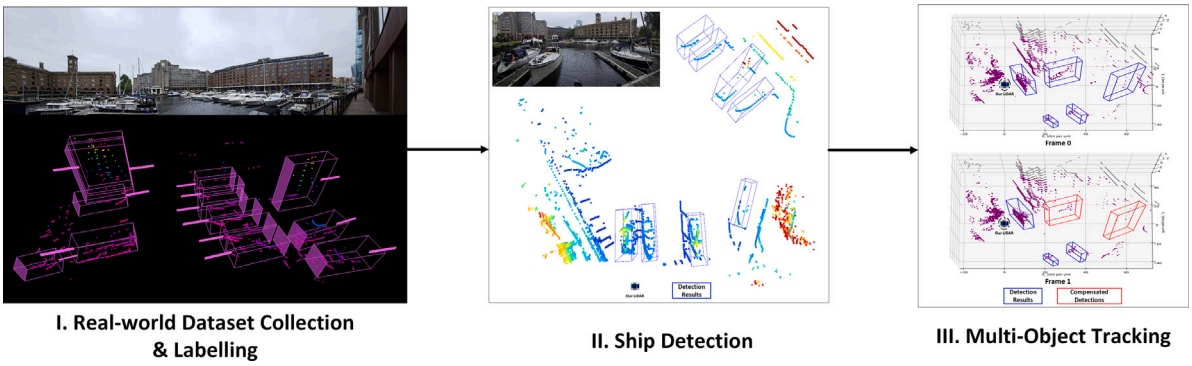


Fig. 2. Proposed framework for ship detection and tracking.

used for ship detection in maritime applications. Compared to the UGV applications, where there are many well-structured datasets such as KITTI (Geiger et al., 2013), CADC (Pitropov et al., 2021) and Hu et al. (2023), most existing detection networks are developed based on these datasets.

2.3. Multi-Object Tracking (MOT)

The roll and pitch motions of the ASVs cause significant changes across the point cloud frames, potentially dividing continuous target objects into several discrete parts. In other words, while a target object might not be detected in the current frame, it is likely that the undetected object in the current frame has been identified in previous frames from an appropriate observation angle. Similarly, occlusion can also cause missed detections. Therefore, relying only on the detection networks may fail to capture continuous target objects, and these discontinuous target objects can affect detection accuracy which may lead to additional safety issues. MOT methods are widely used to overcome such issues. Most existing methods each use particular types of filters to fuse detection results with external sensor data, such as Automatic Identification System (AIS) data, to track targets in maritime applications. Lin et al. (2021) proposed using LiDAR detection results fused with AIS data for tracking objects floating on the water surface using an Adaptive Extended Kalman Filter (AEKF). They later refined their work by using an Extended Kalman Filter (EKF) to improve tracking accuracy (Lin et al., 2022). Chen et al. (2024a) proposed using large-scale AIS data to track and analyse the spatiotemporal distribution of pollutant emissions from ships. Qu et al. (2023) proposed using shore-based cameras to detect ships and fuse the detection results with AIS data using EKF to monitor moving vessels so as to overcome the occlusion problem. In addition to filter-based methods, deep learning-based approaches have been widely used in recent years to continuously track objects. Chen et al. (2024b) proposed an ensemble deep learning framework that incorporates a memory mechanism and uses camera images to track pedestrian trajectories in port areas. This approach effectively compensates for missing trajectory data and removes outliers caused by extreme weather conditions.

It must be noted, however, that AIS data is not always available, such as under bridges or for small ships and buoys that are not equipped to transmit AIS signals. Zhang et al. (2022) proposed using image detection results and an Unscented Kalman Filter (UKF) to track pedestrians. Shan et al. (2021) proposed using LiDAR detection results and a Kalman Filter (KF) to track riverbanks. Given the limitations of AIS data, this research proposes an MOT method that uses only detection results with a KF to track detections that may be missed as a result of ASV motions and occlusions.

3. Methodology

The proposed framework combines ship detection and tracking modules, as shown in Fig. 2. The real-world datasets for the target ships are collected and manually labelled. The processed datasets are then used to train the detection networks. In this framework, frames of raw point cloud data are fed to the ship detection module to generate detection results, including object classes and their corresponding 3D oriented bounding boxes. The target object classes in this research are small boat, medium boat, large boat and buoy. The detection results, particularly the bounding box orientations θ and their x and y coordinates, are then forwarded to the multi-object tracking module to compensate for probable missed detections resulting from the ASV's motions and occlusions. The details of each module are described in the following sections.

3.1. Ship detection

3.1.1. Network architecture

The network used in the ship detection module is switchable, including PointPillar, SECOND, and PV-RCNN. All the networks share a flexible model structure that is easy to modify and supports various networks, as shown in Fig. 3. For the one-stage detection networks, PointPillar and SECOND, the architecture includes a 3D and 2D backbone and a dense head. For the two-stage detection network PV-RCNN, it features an additional Region of Interest (ROI) head.

The 3D backbone includes a Voxel Feature Extractor (VFE) and 3D sparse convolutions. The VFE converts raw point clouds into a structured format that can be processed by networks. For PointPillar, point clouds are divided into vertical pillars during the voxelisation step, while in SECOND and PV-RCNN, space is divided into 3D voxels, as shown in Fig. 4. Due to the sparsity of the point cloud, there might be many empty voxels or pillars. The grid size must be defined based on the LiDAR range. For this research, the LiDAR range is set to -70.4 m to $+70.4$ m in the *x-direction*, -40 m to $+40$ m in the *y-direction*, and -10 m to $+10$ m in the *z-direction*. The corresponding grid size for PointPillar is $(0.2\text{ m}, 0.2\text{ m}, 20\text{ m})$, while the grid size for SECOND and PV-RCNN is $(0.2\text{ m}, 0.2\text{ m}, 0.4\text{ m})$. The 3D sparse convolutions are only implemented in voxel-based methods, as these 3D sparse convolutions are a technique for extracting features and result in a 3D feature map where each voxel contains a feature vector. In other words, instead of processing the entire 3D volume, sparse convolutions focus only on the non-empty (occupied) parts of the grid and their immediate neighbours, skipping the empty regions. This reduces computational processing requirements and memory usage.

For the 2D backbone, the first step is to convert pillars or voxels into a 2D BEV map. For pillars, each pillar contains point features within it, effectively compressing the 3D data into a 2D grid of pillars. Each pillar's features, such as point cloud coordinates x, y, z and reflectance r, along with the distances to the arithmetic mean of all points in the

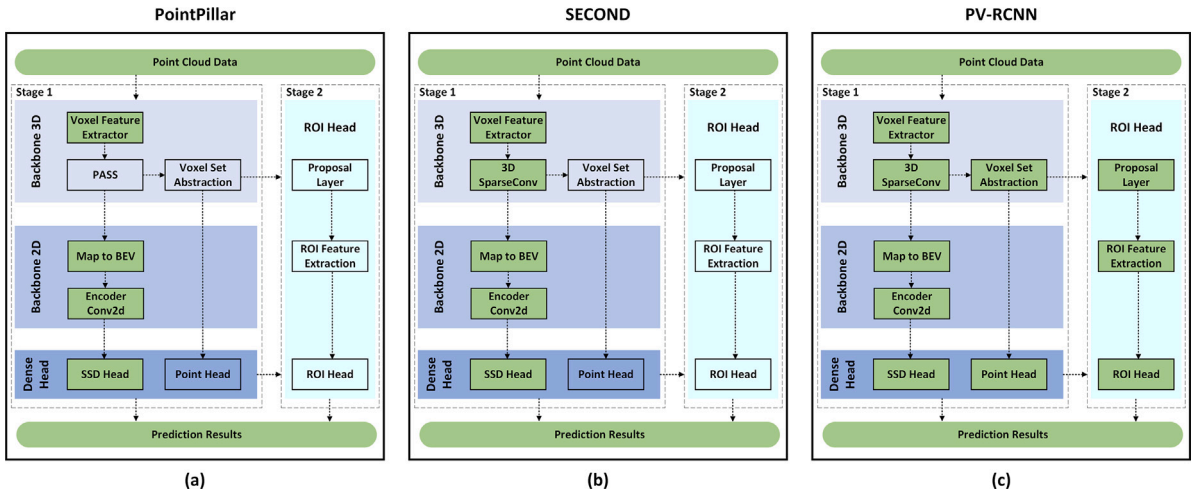


Fig. 3. Detection network architecture for (a) PointPillar, (b) SECOND, and (c) PV-RCNN. The units used in each network are shaded in green.

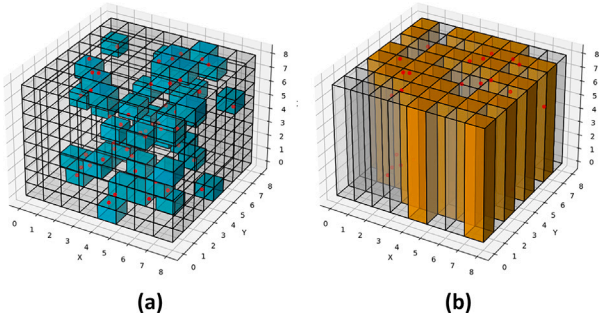


Fig. 4. Voxelisation for (a) voxels and (b) pillars. The red dots represent point clouds, and shaded pillars or voxels represent those containing point clouds.

pillar x_c, y_c, z_c and the offsets from the pillar centre x_p, y_p, are encoded into a feature vector. These encoded features are then projected onto a 2D grid, forming a pseudo-image in BEV. For voxels, the 3D feature map generated by the 3D sparse convolutions is projected onto a 2D grid in the BEV, reducing the dimensionality by collapsing the height dimension while keeping the spatial distribution in the *x*-*y* plane. Each pixel in the BEV pseudo-image contains a feature vector from a pillar or a feature vector derived from the corresponding column of voxels. A 2D CNNs then processes this BEV pseudo-image to perform object detection tasks, exploiting the efficiency of 2D convolutions to extract high-level features and detect objects.

The high-level features extracted by the 2D backbone are then fed to the Single Shot Detection (SSD) Head, which is responsible for the final classification of objects and the regression of their oriented bounding boxes. The SSD Head uses predefined anchors to refine object detection. Anchors, in the context of this report, are predefined bounding boxes of various sizes and aspect ratios that are used as reference points for object detection. They are placed uniformly across the feature map, providing a fixed set of candidate regions where objects might be detected. Each anchor box is associated with a specific position in the feature map and has predefined dimensions. During the detection process, the network evaluates the anchors and adjusts them to better fit the actual objects in the image. This adjustment involves predicting the offsets for the anchors' coordinates and dimensions to match the ground-truth bounding boxes of the objects. The network learns to make these adjustments through the bounding box regression layer in the detection head. The predefined anchors for each class in this research are set based on the statistics of different boat types as shown in Table 3.

For the two-stage networks, such as PV-RCNN, more units work together to refine object detection results. A critical unit is the Voxel Set Abstraction (VSA), which combines voxel features extracted by 3D sparse convolutions with some raw point cloud data that has been randomly selected. The Point Head then refines these point-wise features, classifying the randomly selected points as foreground or background, and helps generate high-quality region proposals. The outputs of the VSA and dense head are forwarded to the second stage to refine the detection results further. In the second stage, the Proposal Layer generates candidate proposals (regions) likely to contain objects using detection results from the SSH head. ROI Feature Extraction involves aggregating the features of each ROI from the randomly selected point features for accurate and robust proposal refinement. These feature vectors are processed by the ROI Head, which performs the final classification of objects and refines the bounding box coordinates for each proposal. The outputs are further filtered using Non-Maximum Suppression (NMS) to produce precise and reliable 3D object detections, combining the computational efficiency of single-stage detection with the accuracy of two-stage refinement.

3.1.2. Datasets

The datasets used in this research include 10,000 simulated datasets (5000 frames with ship point clouds only and 5000 frames with both ship and terrain point clouds) and 9200 real-world datasets. The datasets include various types of buoys and ships, such as ships with and without sails, monohulls, and catamarans, with boat sizes ranging from 3 m to 40 m. The simulated dataset, proposed by Lin et al. (2022), has been refined to address issues where ground-truth oriented bounding boxes were incorrectly labelled, particularly in scenarios involving terrain point clouds. Additionally, some boats with fewer features have been removed, as it cannot be expected that the detector accurately detect targets with a limited number of point clouds, as shown in Fig. 5.

Initially, the detection networks were trained using only the simulated datasets and tested on some real-world data without any labelling. However, the detection performance was not ideal, as discussed in Section 1. Consequently, more real-world data from diverse scenarios was collected, including ships harboured in marinas and on the River Thames, as well as ships moving in marinas and on the River Thames, with all data being labelled.

It is important to note that the labelling process for point cloud data is much more involved than that for image data, as point cloud data is more abstract. Therefore, the ground-truth labels across frames can differ slightly. For the simulated dataset, the target classes are divided into small boats (length less than 7 m), medium boats (length ranging

Table 3
Predefined anchors for simulated datasets and real-world datasets. Anchor rotation refers to the orientation adjustments of anchor boxes to better match the possible orientations of objects. The matched and unmatched thresholds are values that determine when an anchor box is considered a positive or negative match for a ground-truth bounding box based on IoU.

Classes	Anchor size $(Length, Width, Height)$ (m)	Anchor rotations (rad)	Anchor bottom height (m)	Matched threshold (%)	Unmatched threshold (%)
Real − world datasets					
Small boat	(8.68, 2.67, 6.77)	[0, 1.57]	−4.71	50	35
Medium boat	(14.11, 4.54, 12.01)	[0, 1.57]	−5.35	50	35
Large boat	(28.89, 6.36, 10.19)	[0, 1.57]	−6.33	60	35
Simulated datasets					
Small boat	(4.28, 1.92, 1.60)	[0, 1.57]	−2.08	50	35
Medium boat	(9.39, 3.15, 7.54)	[0, 1.57]	−2.83	50	35
Large boat	(15.74, 4.40, 15.92)	[0, 1.57]	−3.62	60	35
Buoy	(0.93, 0.93, 1.47)	[0, 1.57]	−2.04	45	30

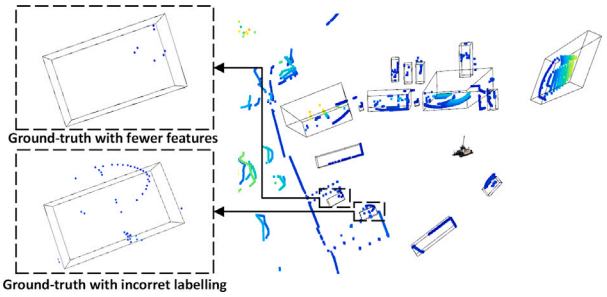


Fig. 5. An example of a simulated dataset with incorrect labelling (point clouds of the ship are outside the bounding boxes) and ground-truth with fewer features (only 7 point clouds within the oriented bounding boxes).

from 7 m to 13 m), large boats (length greater than 13 m) and buoys. For the real-world dataset, the classes are similar to those in the simulated datasets but with different parameters to distinguish boat classes, as the range of boat lengths in the collected real-world data is much broader. The overall datasets are divided into 85% for training and 15% for evaluation.

To address the challenge of limited datasets, a data augmentation method, which involved global flipping, scaling, and rotation applied to the entire point cloud as well as all ground-truth bounding boxes, was introduced. In this particular case, a scaling factor was sampled from a uniform distribution in the range $[0.95, 1.05]$, while a rotation factor was sampled from the interval $[-\pi/4, +\pi/4]$.

3.1.3. Network training

All the networks were trained for 120 epochs with a batch size of 6 for simulated datasets and 5 for real-world datasets. All the detectors were trained using stochastic gradient descent (SGD). An Adam optimiser with a maximum of 100 point clouds within the pillar or voxel was employed. The initial learning rate was 0.0002, with the learning rate schedule using the one-cycle policy (Smith, 2018).

The loss functions introduced in PointPillar, SECOND and PV-RCNN detectors are the same as in the SECOND paper (Yan et al., 2018). The oriented bounding boxes are defined as x, y, z, l, w, h, θ, where x, y, z are the coordinates of the box centre, l, w, h are the length, width, and height of the boxes, and θ is the yaw rotation of the boxes (ships). The regression residual between the anchors and ground-truth is defined by:

$$\Delta x = \frac{x^{gt} - x^{a}}{d_a} \tag{1a}$$

$$\Delta y = \frac{y^{gt} - y^{a}}{d_a} \tag{1b}$$

$$\Delta z = \frac{z^{gt} - z^{a}}{h_a} \tag{1c}$$

$$\Delta w = \log \frac{w^{gt}}{w^{a}} \tag{1d}$$

$$\Delta l = \log \frac{l^{gt}}{l_a} \tag{1e}$$

$$\Delta h = \log \frac{h^{gt}}{h_a} \tag{1f}$$

$$\Delta \theta = \sin(\theta^{gt} - \theta^{a}) \tag{1g}$$

where subscripts gt and a represent ground-truth and anchor, respectively, and $d_a = \sqrt{(w_a)^2 + (l_a)^2}$ is the diagonal of the base of the anchor box.

The total localisation loss is calculated using the smooth L1 loss function as follows (Lang et al., 2019):

$$\mathcal{L}_{\text{loc}} = \sum_{i=1}^{n} \left(\sum_{b_i \in \{B\}} \text{SmoothL1}(b_i) \right) \tag{2}$$

where n is the number of ground-truths, and $B = \{\Delta x, \Delta y, \Delta z, \Delta w, \Delta l, \Delta h, \Delta \theta\}$.

The above methods cannot directly distinguish flipped boxes for yaw angle regression. Therefore, an additional Softmax classification loss is applied to the discretised directions to enable the network to learn the heading $\mathcal{L}_{\text{dir}}$.

$$\mathcal{L}_{\text{dir}} = -\sum_{i=1}^{n} \theta_{gt_i} \cdot \log \left(\frac{1}{1 + \exp(-\theta_{a_i})} \right) \tag{3}$$

We use the standard focal loss for object classification (Lin et al., 2020b):

$$\mathcal{L}_{\text{cls}} = -\sum_{i=1}^{n} \alpha (1 - p_{t_i})^{\gamma} \log p_{t_i} \tag{4}$$

where p_t is the model's estimated probability for the true class. $\alpha = 0.25$ is a balancing factor to address class imbalance. $\gamma = 2$ is a focusing parameter that adjusts the rate at which easier-to-detect objects are given reduced weighting, the parameters chosen are referenced from Lang et al. (2019).

The total loss is therefore:

$$\mathcal{L} = \beta_{\text{loc}} \mathcal{L}_{\text{loc}} + \beta_{\text{cls}} \mathcal{L}_{\text{cls}} + \beta_{\text{dir}} \mathcal{L}_{\text{dir}} \tag{5}$$

where, $\beta_{\text{loc}} = 2$, $\beta_{\text{cls}} = 1$, and $\beta_{\text{dir}} = 0.3$ are the constant coefficients of the total loss, used to balance the respective loss components. The constant coefficients depend on the importance of each detection result. In this context, classification accuracy is not a primary focus since the ASV needs to avoid all detected ships, regardless of their class. Therefore, we set the classification coefficient to 1 as normal. However, the location and size of the bounding boxes are crucial and more emphasis on location losses is required. The heading direction in the context of object detection is not critical, so it is set to 0.2. If the focus was on predicting the motion of ships, then the heading classification would be of greater import and this parameter would have a higher setting.

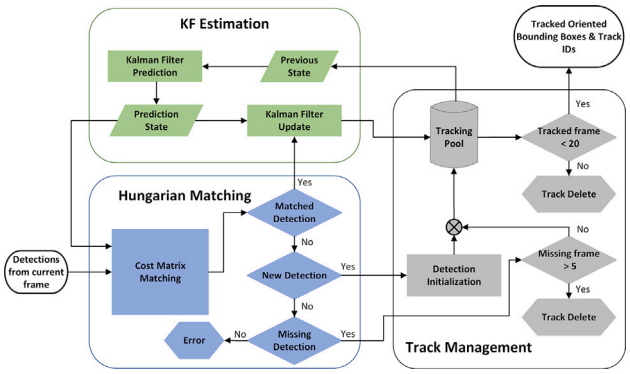


Fig. 6. Multi-object tracking framework.

3.1.4. Evaluation

The evaluation metric for 3D object detection is similar to 2D detection using Average Precision (AP), which is the area under the Precision–Recall (PR) curve. However, in the 3D scenario, we calculate the AP for both the 3D bounding boxes and BEV bounding boxes. Unlike the well-known point cloud datasets such as KITTI (Geiger et al., 2013), truncation and occlusion rates are not considered in this evaluation. The process begins by calculating the 3D Intersection over Union (IoU) between predicted and ground-truth bounding boxes. Next, an empty score set S is created. For each ground-truth bounding box, the unmatched predicted bounding box d_b with the highest IoU greater than a specified threshold is selected as the matching box, and its prediction score (model's estimated probability) is added to S. If no d_b meets the criteria, no score is added to S. The scores in S are then sorted in descending order, forming a set of threshold scores, S^*. For each threshold in S^*, True Positives (TP), False Positives (FP) and False Negatives (FN) are calculated. For each threshold, precision and recall are calculated, where precision is defined as $\frac{TP}{TP+FP}$, and recall is defined as $\frac{TP}{TP+FN}$. The PR curve is plotted using these precision and recall values, and the area under the PR curve is computed to obtain the AP. The overall performance of the detector is measured by the mean AP (mAP), which is the average of the AP values for each class.

3.2. Multi-Object Tracking (MOT)

Algorithm 1 KF-based Multi Object Tracking

Input: Current frame detections, Q_t
Output: Tracked objects in the current frame, Q_{tracked}

1: **Matching:** $Match \leftarrow Hungarian(Q_t, Q_{tracked,t-1})$
2: **for** each $q \in Q_t$ and $p \in Q_{\text{tracked},t-1}$ **do**
3: **if** $Match(q) = \text{NULL}$ **then**
4: $new_object \leftarrow Kalman_Initialize(q)$
5: $trackID \leftarrow trackID + 1$
6: **Append** new_object into Q_{tracked}
7: **else if** $Match(q) = p$ for some $p \in Q_{\text{tracked},t-1}$ **then**
8: $matched_object \leftarrow Kalman_Correct(p, q)$
9: **Append** $matched_object$ into Q_{tracked}
10: **else if** $Match(p) = \text{NULL}$ **then**
11: **if** $missing_match < 3$ **then**
12: $missing_object \leftarrow Kalman_Predict(p)$
13: $missing_match \leftarrow missing_match + 1$
14: **Append** $missing_match$ into Q_{tracked}
15: **end if**
16: **end if**
17: **end for**

The proposed MOT module integrates a KF (Kordic and Kordic, 2010), Hungarian algorithm (Kuhn, 2005) and a track management strategy, as shown in Fig. 6. The Hungarian algorithm correlates the current detection results with historical detections. The KF initialises and predicts the detections, while the track management strategy processes the objects in accordance with their assigned IDs. If a new object is detected in the current frame, it is initialised by a KF and assigned a new ID. If the current detection matches with a previous detection, the existing KF is updated. If there is a detection missing from within the current frame, the historical detection results are used to predict the position of the missing detection in the current frame. If a detection is missed for a certain number of frames, it is deleted. Details are outlined in Algorithm 1.

The process begins by pairing new detections Q_t with those from the previous frame $Q_{\text{tracked},t-1}$ using the Hungarian algorithm, which relies on the Euclidean distance between object pairs for matching. For new detections, as described in lines 3–6, initialise each detection using a KF, assign a new tracking ID, and return the results to the tracking pool $Q_tracked$, which stores all the historical detections.

Due to the challenges posed when trying to accurately model the dynamics of target ships, the employed KF uses the simplest model for target tracking, which is the white-noise constant velocity (CV) model (Bar-Shalom et al., 2004). This model assumes that target speed is an independent process driven by Gaussian white noise. The CV model is sufficient to predict detection results over short periods because the frequency of LiDAR output can reach up to 20 Hz, reducing possible error accumulation through rapid updating. The employed KF is as follows:

$$X(t) = AX(t-1) + \omega(t-1) \tag{6}$$

$$Z(t) = HX(t) + v(t) \tag{7}$$

$$X = \begin{bmatrix} x \\ y \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}, \quad Z = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 0 & 0 & T & 0 & 0 \\ 0 & 1 & 0 & 0 & T & 0 \\ 0 & 0 & 1 & 0 & 0 & T \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}, \quad P = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

where, X is the state vector, which includes x, y, θ, $\dot{x}$, $\dot{y}$, and $\dot{\theta}$. These represent the x and y coordinates of the detection box's centre, the heading angle θ, and the corresponding velocities $\dot{x}$, $\dot{y}$, and $\dot{\theta}$. Z is the measurement vector. A is the state transition matrix, with a sampling time T, and H denotes the state-to-measurement matrix. Both w and v are zero-mean Gaussian noise covariance matrices, representing process and measurement uncertainties, with values of 10^{-3} and 10^{-1}, respectively, on the diagonal. P is the state covariance matrix.

For $Q_{\text{tracked},t-1}$ that match with Q_t (line 7–9), the KF is utilised to update the prior frame's predictive model with the latest measurements, Eq. (7). Additionally, for objects not detected in the current frame but observed in earlier frames (line 10–14), predictions from the last frame are used to compensate for these missing detections Eq. (6). Objects that have been missed for more than a set number of frames are excluded from tracking. This methodology provides a framework for tracking multiple objects across sequences of frames, effectively overcoming the challenges that arise from discontinuous detection caused by the maritime environment.

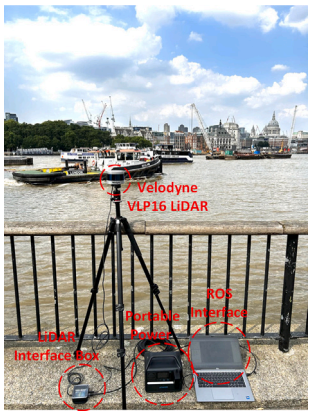


Fig. 7. LiDAR setup for the real-world data collection.

4. Results

The proposed ship detection and tracking framework was implemented on a server with an AMD EPYC 7T83 CPU and an NVIDIA RTX 4090 GPU. All the detection networks were trained and evaluated in the PyTorch environment. The experiment was first conducted on the modified simulated datasets to compare the performance of each type of network, as mentioned in Section 3.1.1. The experiment was then conducted on the real-world datasets to evaluate the performance of the proposed framework in real-world applications.

4.1. Real-world datasets collection

The real-world dataset was collected across a busy maritime environment, including the marina and the River Thames. Due to the regulations on operating ASVs in such busy areas, an ASV could not be driven directly around the location to collect the dataset. The alternative solution was to position the LiDAR near the bank and manually move the LiDAR around to simulate the scenario of a moving ASV. The LiDAR setup is shown in Fig. 7. A Velodyne VLP-16 LiDAR with 16 channels and a measurement range of up to 100 m was employed. The accuracy is typically up to ±3 cm, with a field of view of ±15°. The rotation rate is set to 15 Hz, which can collect approximately 15 frames per second. The LiDAR is interfaced with ROS to view the data in real-time and record the data in the format of a ROS bag for further processing.

The collected data was then labelled manually using commercial software, *Supervisely*. The target ships are first labelled and put into one class, *boat*, and then divided into more classes based on the size of the ship. The labelling process is time-consuming, and compared to images, point cloud data is more abstract. Additionally, the bounding box for the point cloud data is 3D, but the LiDAR cannot capture all the features of the ship, usually just part of it. This means it can be difficult to accurately define the bounding box around the target ship at the correct size and orientation, even for experts. Moreover, unlike vehicles running on solid surfaces, the heave motion of the ship can also make it difficult when setting the vertical position of the bounding boxes. Considering the above challenges, we developed a solution to find the target ships across the frames that have the richest features, specifically those that capture features from at least two sides of the ship. Based on this, we determine the size of the bounding box and apply it to each frame. Because point cloud data, unlike images, do not change the size of the ship with distance, this method ensures the consistency of the bounding box for each type of ship. Additionally, the height of the bounding boxes were extended to allow for some margin to address the heave motions. Examples of the data collected with labels are shown in Fig. 8.

The processed dataset is managed in the same way as the KITTI datasets (Geiger et al., 2013), as shown in Fig. 9, to ensure good generalisation from our collected datasets. This is because most point cloud-based detection networks are developed based on the KITTI datasets. We have kept the calibration files for calibrating the camera and LiDAR, and the image files, to make it easier to integrate camera and LiDAR data in the future, if ever necessary. For now, the image files contain images with white noise, and the values in the calibration files are all set to zero to maintain consistency with the KITTI datasets. The Imagesets folder contains the sequence numbers of the frames that are separated for training, validation, and testing. To prevent overfitting in our trained model, we have deliberately selected frames for training that are distinct from those used for testing and validation, featuring different scenes rather than just randomly separating the datasets. This approach is necessary because the LiDAR can collect up to 30 frames per second, meaning the differences across sequences of frames are minimal.

4.2. Ship detection results

The detection network PointPillar, used as our baseline model, was initially only trained on simulated datasets and then evaluated using both simulated data and a small amount of real-world data. When evaluated using real-world data, the PointPillar offered very poor performance, as shown in Fig. 1, with the highest mAP being 29.8%. This indicated that the detection network could not effectively interpret the real-world input frames. However, its detection performance on simulated data was excellent. The primary issue was the poor generalisation of the simulated datasets used for training the networks, which motivated us to collect real-world datasets specifically for maritime applications.

Subsequently, PointPillar was trained using a combination of both simulated and real-world datasets. This approach yielded mAP performance similar to the networks trained with real-world datasets only. However, by inspecting the visualised detection results, it was found that there were a slightly higher number of incorrect detections, which mislabelled some terrain point clouds as targets, as shown in Fig. 10. Still, compared to the detection performance of the network trained with only simulated data, as shown in Fig. 1, it showed notable progress. This problem occurred because the ground-truth in the simulated datasets included many targets with sparse or minimal features. Despite refining the simulated datasets by removing ground-truths with fewer than 15 point clouds within the bounding boxes, this was still insufficient. Additionally, the training time for the combined datasets was significantly longer, increasing from approximately 9 h to 20 h. Given these findings, it was decided to train the remaining detection networks separately on simulated datasets and real-world datasets to optimise performance and training efficiency.

4.2.1. Detection results based on simulated datasets

The detection performance for each detection network trained and tested on simulated datasets is presented in Table 4. Following the removal of incorrectly labelled ground-truth data, the elimination of ground-truth data with a limited amount of features, and the addition of a new large boat class. The PointPillar results demonstrated an overall improvement of 7.2% in detection performance compared to the results reported in the original paper that introduced these simulated datasets. Furthermore, PV-RCNN, which offered the best detection accuracy, showed an overall improvement of 25.3% compared to the performance reported in the original paper. Fig. 11 shows examples of visualised detection results based on the simulated dataset. The detection results are generally good; however, there are still some detection inaccuracies. As shown in Fig. 11(b), the bounding box does not correctly include all the point clouds of the target ship, and there are some missed detections, as shown in Fig. 11(d), where some ships with rich features are still not detected. However, these incorrect and missed detections rarely

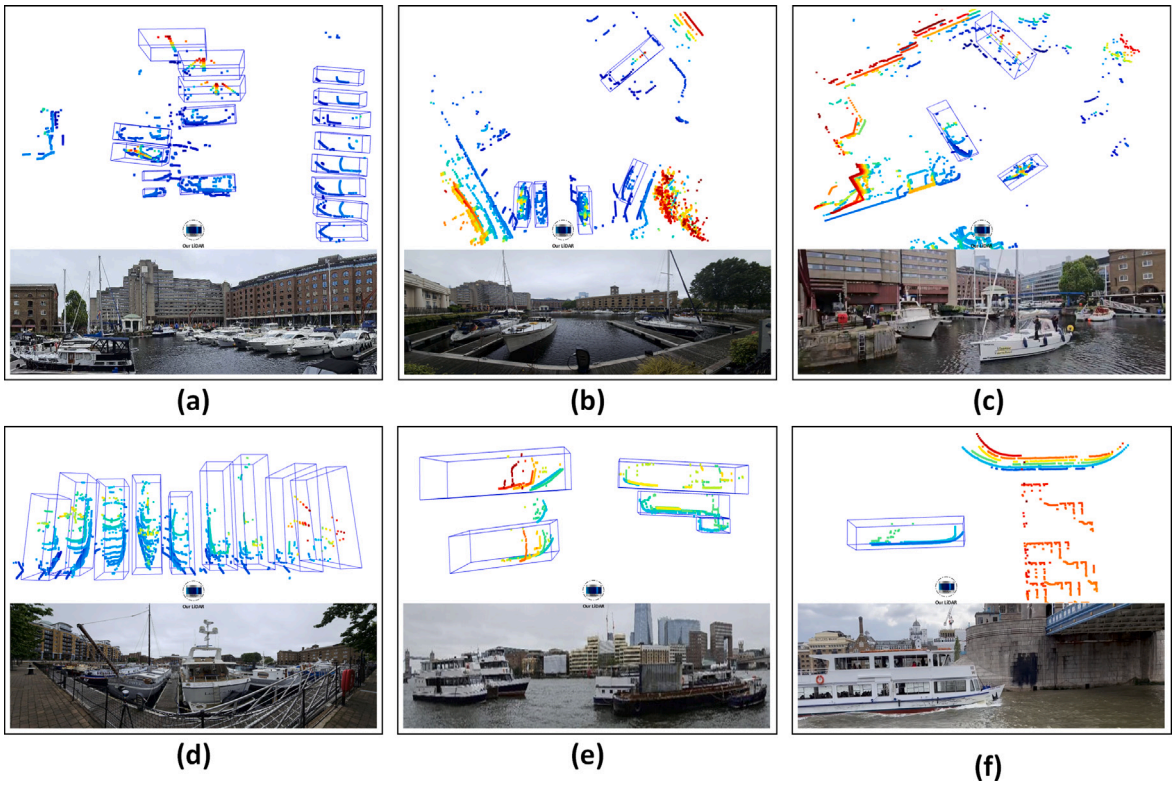


Fig. 8. Examples of collected data: (a), (b), and (d) are ships harboured in the marina; (e) is a ship harboured in the Thames River; (c) and (f) are ships moving in the marina and the Thames River, respectively.

Table 4
Evaluation results for each detection network. The networks are trained and evaluated separately on simulated and real-world data. FP_{ML} refers to the detection results that are mislocalised but correctly classified, while FP_{BG} refers to the detection results that are either not localised or not classified correctly.

Detection networks	Small boat AP (%)	Medium boat AP (%)	Large boat AP (%)	Buoy AP (%)	mAP (%)	TP	FP_{ML}	FP_{BG}	Inference time (ms/frame)
Simulated datasets									
PointPillar (Lin et al., 2022)	52.4	70.7	–	47.1	**60.8**	–	–	–	–
PointPillar (ours)	62.0	69.0	77.8	51.8	**65.2**	15,866	3670	2998	19.4
SECOND	69.9	73.7	77.9	70.5	**73.0**	16,829	2527	2545	26.3
PV-RCNN	76.7	75.1	78.0	75.1	**76.2**	17,007	2160	900	39.5
Real-world datasets									
PointPillar	62.1	63.7	64.2	–	**63.3**	1925	491	101	8.4
SECOND	68.7	72.3	68.8	–	**69.9**	2015	579	170	15.6
PV-RCNN	83.2	76.3	62.7	–	**74.1**	2167	366	151	43.3

occur, as shown by comparing the ground-truth numbers and detected numbers across each frame.

Comparing the performance between the detection networks introduced in this research, the results align with the statement in Section 2.1, where PointPillar has the fastest inference time, followed by SECOND and PV-RCNN. In terms of detection accuracy PV-RCNN ranks highest followed by SECOND and PointPillar. Interestingly, the detection accuracy for small targets, such as small boats less than 7 m and buoys with an average area of less than 1 m^2, showed the most significant improvement across all detections. For example, the detection accuracy for small boats increased by 12% when comparing PointPillar to SECOND, but the detection accuracy for large boats only increased by 0.13%. This is due to the data processing methods; small targets naturally have fewer features than larger targets, and after extracting high-level features from 3D point clouds to 2D pseudo-images, important features in small targets are more likely to be lost. Voxel-based methods, compared to pillar-based methods, are more likely to retain the original features, resulting in SECOND having better overall detection performance than PointPillar. Additionally, PV-RCNN predicts the ROI to refine the final detection results, further improving accuracy.

4.2.2. Detection results based on real-world datasets

The detection performance of each of the networks trained and tested with real-world datasets is shown in Table 4. The mAP results for PointPillar trained with the real-world dataset show an improvement of 112% compared to when the training was carried out with simulated datasets, based on real-world evaluations. The mAP results for PV-RCNN are the highest at 74.1%. The visualised results are shown in Fig. 12. The detection results are generally good when the target ships have rich features; however, we found that all the detection networks are likely to incorrectly label target ships with fewer point cloud features. As shown in Fig. 12(b), the ship in the red circle has very limited features, but the networks still attempted to label it, leading to an error in predicting the bounding box's heading, which should be as indicated by the arrow. Considering that most ground-truths with reduced features are not labelled in the labelling process but are still detected by the networks, it is possible that the detection network

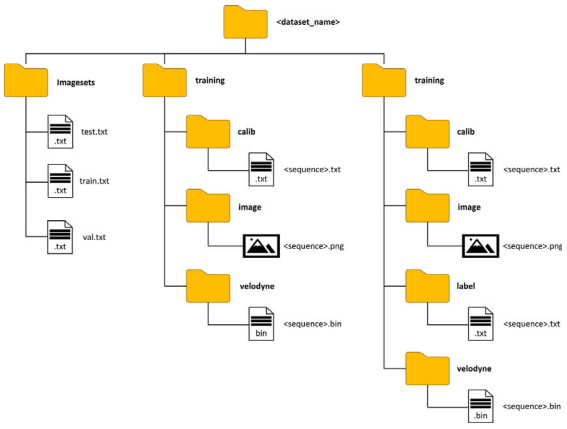


Fig. 9. Dataset structure.

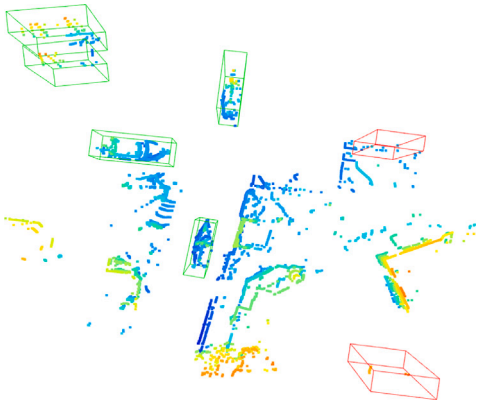


Fig. 10. Visualised detection results for the real-world data with detection networks trained on combined simulated and real-world datasets. The green bounding boxes show the correct detections, while the red bounding boxes show the incorrect detections.

is exhibiting some over-fitting problems. A possible solution to this problem is to integrate the networks under investigation with Bayesian networks, as the Bayesian network can provide an uncertainty measurement of the detection results, meaning that the networks can indicate a level of confidence in relation to the results. Incorrect detections can be rationally eliminated using this approach as they would be registered as having high uncertainty.

Furthermore, for large ships with a length of more than 30 m, such as the Uber Boat, incorrect detection results are more likely. As shown in Fig. 12(e) and (h), the bounding box does not include all the point clouds that belong to the target ships, as indicated by the red circles. It is important to note that these incorrect detections are particularly dangerous for the ASV. Although it may appear that only a small part of the ship is outside the bounding boxes, considering the size of these ships, the missing parts could actually be around 5 m in length. Given that the ASV is usually about 3 m in length, this poses a significant hazard. Additionally, it could do significant damage to the ship, such as badly damaging a propeller or rudder.

It can, however, be argued that since the relevant large ship appears in fewer than 800 frames, the current detection results for these large ships, with such limited training data, are already impressive, showing the potential of LiDAR-based ship detection methods. The solution to these issues is relatively straightforward but there are challenges. There would be need to collect more real-world data from different scenarios and with various types of ships, refining the detection networks, or setting specified safety margins.

Overall, the performance of the introduced detection networks has been validated and met expectations in real-world testing. However,

Table 5
Implementation strategy for different scenarios. The scenarios are classified based on the available navigable area and the inference speed requirements for the detection networks. The marina, as a baseline scenario, has the smallest navigable area and requires the lowest inference speed. The scenario examples are (a) busy marina, (b) busy channel, and (c) open water area.

Scenario description	Navigable area	Inference speed requirements	Detection network
Marina *(a)*	Small	Slow	PV-RCNN
Channel *(b)*	Medium	Medium	SECOND
Open water *(c)*	Large	Fast	PointPillar

Scenario examples



due to the complexity of maritime traffic, it is not straightforward to determine which detection network is best suited for maritime perception. Therefore, the modularised network architecture is advantageous, as it offers the flexibility to easily switch between different detection networks to adapt to various maritime traffic conditions. The implementation strategy has been developed based on the results of extensive real-world testing, as shown in Table 5.

In a busy marina scenario, the navigable area is typically small, requiring much higher detection accuracy. However, since ships in the marina usually move slowly, the inference speed requirements for the detection networks are not as critical. Therefore, PV-RCNN is the preferred detection network for such scenarios. In contrast, for a busy shipping channel like the River Thames, the navigable area is much larger, and the ships move much faster. Thus, SECOND may be more appropriate as it balances detection accuracy and inference speed. In open water scenarios, such as offshore areas, the navigable area is the largest, and ships tend to move relatively quickly. Therefore, inference time needs to be prioritised, even if it means slightly compromising detection accuracy, making PointPillar the ideal detection network for such scenarios.

4.2.3. Probabilistic object detection

In unstructured and complex maritime environments, detection results can exhibit varying degrees of accuracy, leading to higher failure rates. The introduced ship detection networks may be experiencing overfitting issues, resulting in occasional incorrect and missed detections, which pose significant safety risks for ASVs. Identifying reliable perceptual uncertainties, which indicate detection inaccuracies or sensor noise, is crucial for evaluating detection performance and enabling ASVs to respond appropriately. However, all the introduced detection networks operate deterministically, providing detection results without indicating their level of uncertainty.

To address these issues, the concept of probabilistic modelling can be integrated into the introduced detection networks. An experiment was conducted to test the feasibility of this approach. In the experiment, PointPillar was trained four times separately with different initial weightings. The detection results for each PointPillar vary due to the differences in their weightings after training, as shown in Fig. 13(a). For target ships with rich point cloud features, the detection results are relatively similar, but for target ships with fewer point cloud features, each detection result shows significant differences. Additionally, occasional detection errors occur due to background noise. The detection results from each network can be combined to produce a predictive distribution, which measures the overall detection uncertainty. Fig. 13(b) illustrates the concept of probabilistic ship detection.

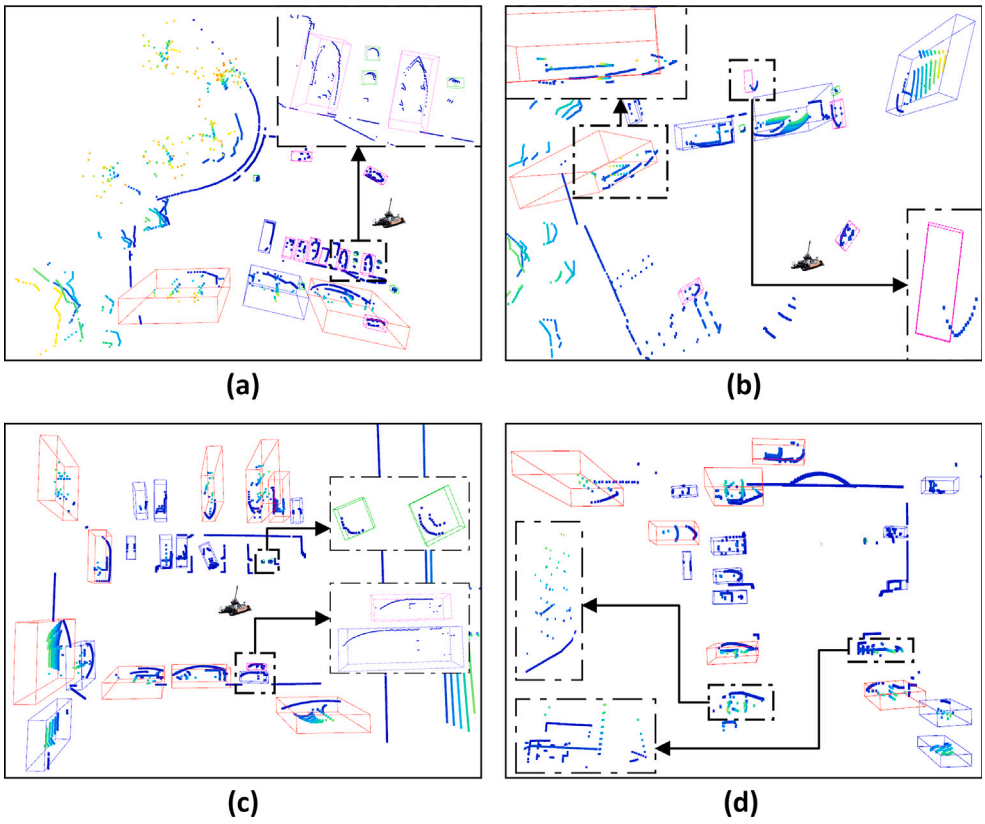


Fig. 11. Visualised detection results for the simulated dataset. (a) and (c) show frames with excellent detection results, while (b) and (d) show frames with slightly incorrect detection. Red bounding boxes indicate large boats, blue bounding boxes indicate medium boats, violet bounding boxes indicate small boats, and green bounding boxes indicate buoys.

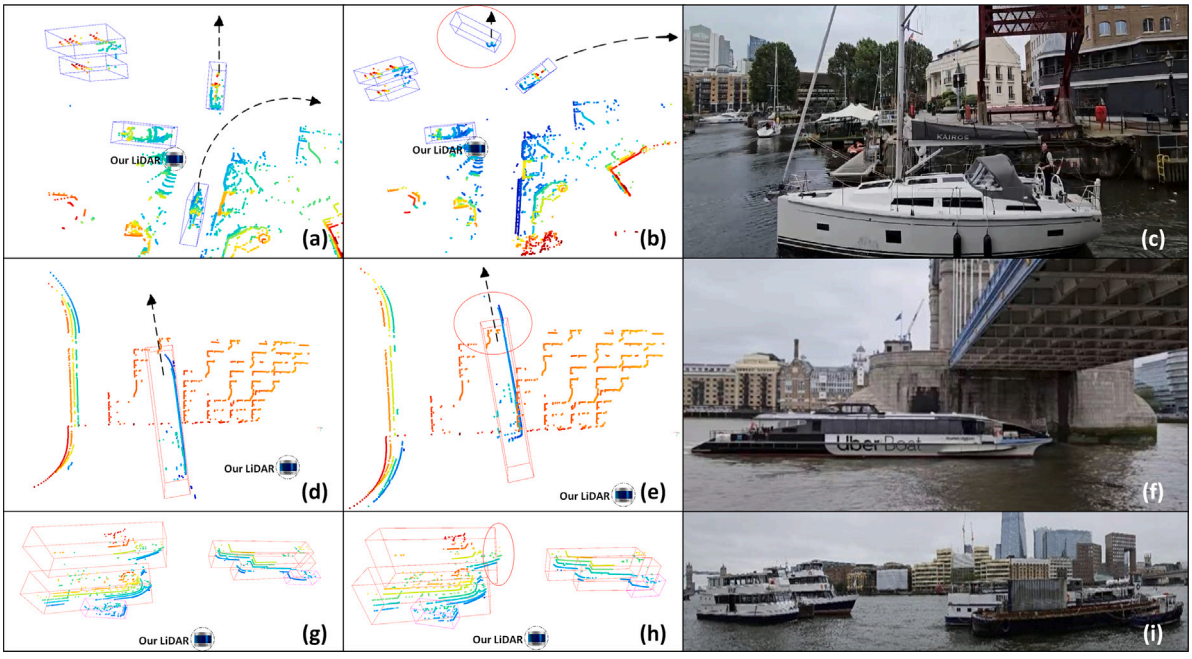


Fig. 12. Visualised detection results for the real-world dataset. (a), (d), and (g) show frames with excellent detection results across different scenarios, while (b), (e), and (h) show frames with slightly incorrect detection. (c), (f), and (i) are the images for the corresponding scenarios. Red bounding boxes indicate large boats, blue bounding boxes indicate medium boats, violet bounding boxes indicate small boats, and green bounding boxes indicate buoys. The red circles highlight areas of incorrect detections.

Green bounding boxes indicate that the detection networks are certain about their results. Orange bounding boxes represent detection results with moderate uncertainty; in this case, although the ship is correctly detected, the limited point cloud features may lead to mislocation and misclassification. Red bounding boxes indicate detection results with high uncertainty; in this scenario, terrain point clouds have been incorrectly detected as ships. Future work will focus on developing and implementing probabilistic detection networks.

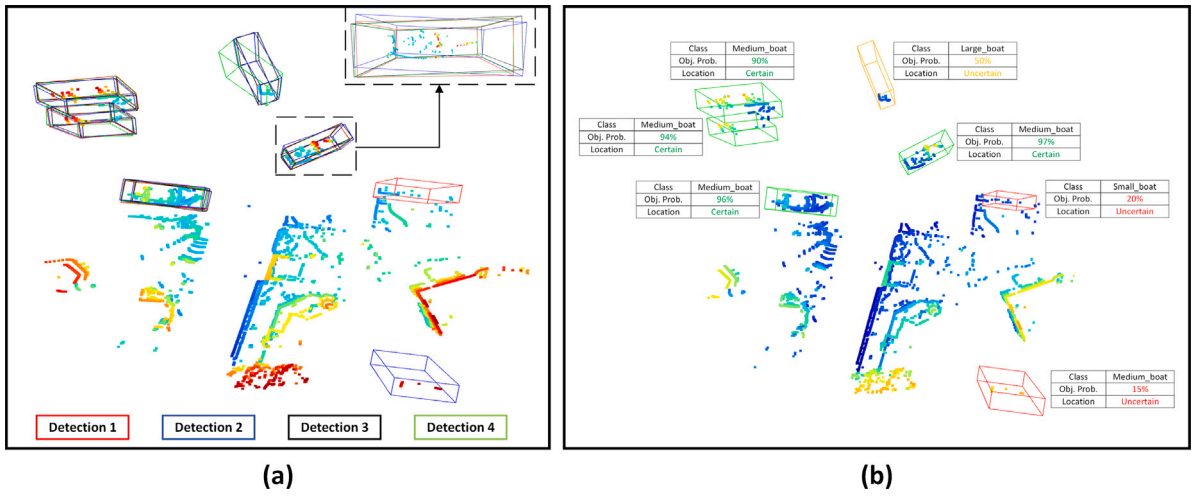


Fig. 13. Probabilistic ship detection. (a) Experimental results, where each colour represents the detection results from a specific network. (b) A conceptual illustration of probabilistic ship detection: Each object is classified with a probability, and its bounding box is predicted with an uncertainty interval.

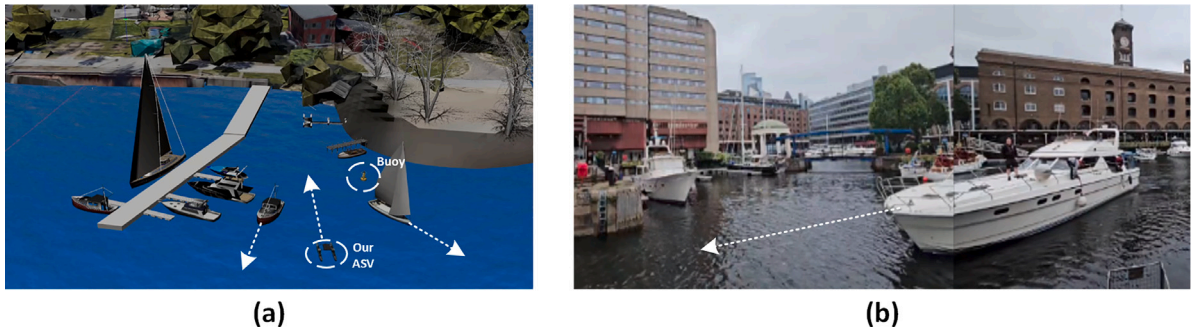


Fig. 14. Scenarios for MOT testing. (a) Simulated scenario (b) Real-world scenario. The white arrow indicates the ship moving in the direction of the arrow.

4.3. Multi-object tracking results

The proposed MOT methods were tested in both simulated and real-world scenarios. The results from each testing scenario were quite similar, so we have selected two scenarios that best illustrate the features of our proposed MOT methods, as shown in Fig. 14. The details of these two selected scenarios are described below. In the simulated scenario, the ASV equipped with LiDAR is moving in the direction indicated by the arrow. There are also six different types of ships, with four harboured, two moving, and one buoy. In the real-world scenario, the LiDAR is fixed near the bank and has four ships in its range, with three being harboured and one in transit.

The tracking results for the simulated scenarios are shown in Fig. 15(a). Due to the high output frequency of the LiDAR, changes across adjacent frames are difficult to distinguish. Therefore, we have selected some frames with significant changes to describe the performance of the MOT modules. From frame 0 to frame 100, the LiDAR collected rich features for the target ship, making it easy for the detection networks to correctly detect them. Note that in frame 50, the two ships were not detected because they were out of the detection range. The detection range is ±40 m in the *y-direction*, but the point cloud collected was outside this range, shown as grey points. However, the MOT modules successfully compensated for these missed detections based on historical detection results.

From frame 150 to frame 180, due to occlusion, the point cloud features for the ship in the red bounding boxes, located approximately at (20 m, 40 m), became less visible. Despite this, the ship was successfully tracked, except in frame 180, where the missing ship was not compensated for as the ship had not been detected for such an extended period that it was already eliminated. To prevent accumulated tracking errors, we stopped tracking that ship. Note that the compensated ship in frame 180, located at approximately (10 m, 20 m), is different from the one in frames 150 and 170. From frame 200 onwards, the compensated ships were no longer occluded and had more point cloud features, allowing them to be successfully detected. Again, in frame 200, the ship actually had rich features but was missed. This only happens when using the simulated datasets, as discussed in Section 4.2.1. Based on the historical detection results, such situations can be easily prevented if they occur only occasionally.

The tracking results for the real-world scenarios are similar to the simulated scenarios, as shown in Fig. 15(b). Additionally, because the LiDAR is positioned on the bank, we cannot assess how well the proposed MOT module compensates for missing detections caused by the roll and pitch motions of the ASV. In general, our proposed MOT module is reliable in compensating for missing detections, relying solely on the detection results without any extra measurements, such as AIS. Continuous detection results across frames can further help ensure the safe operation of the ASV and effectively plan the path for collision free manoeuvring.

5. Conclusion

In this paper, we discussed the challenges faced by maritime environmental perception and the drawbacks of existing methods. We proposed a reliable LiDAR-based ship detection and tracking framework for busy maritime environments. By employing a modularised network structure, we were able to switch between different detection networks to meet various task requirements. The networks, including PointPillar, SECOND, and PV-RCNN, were trained and evaluated using both simulated and real-world datasets.

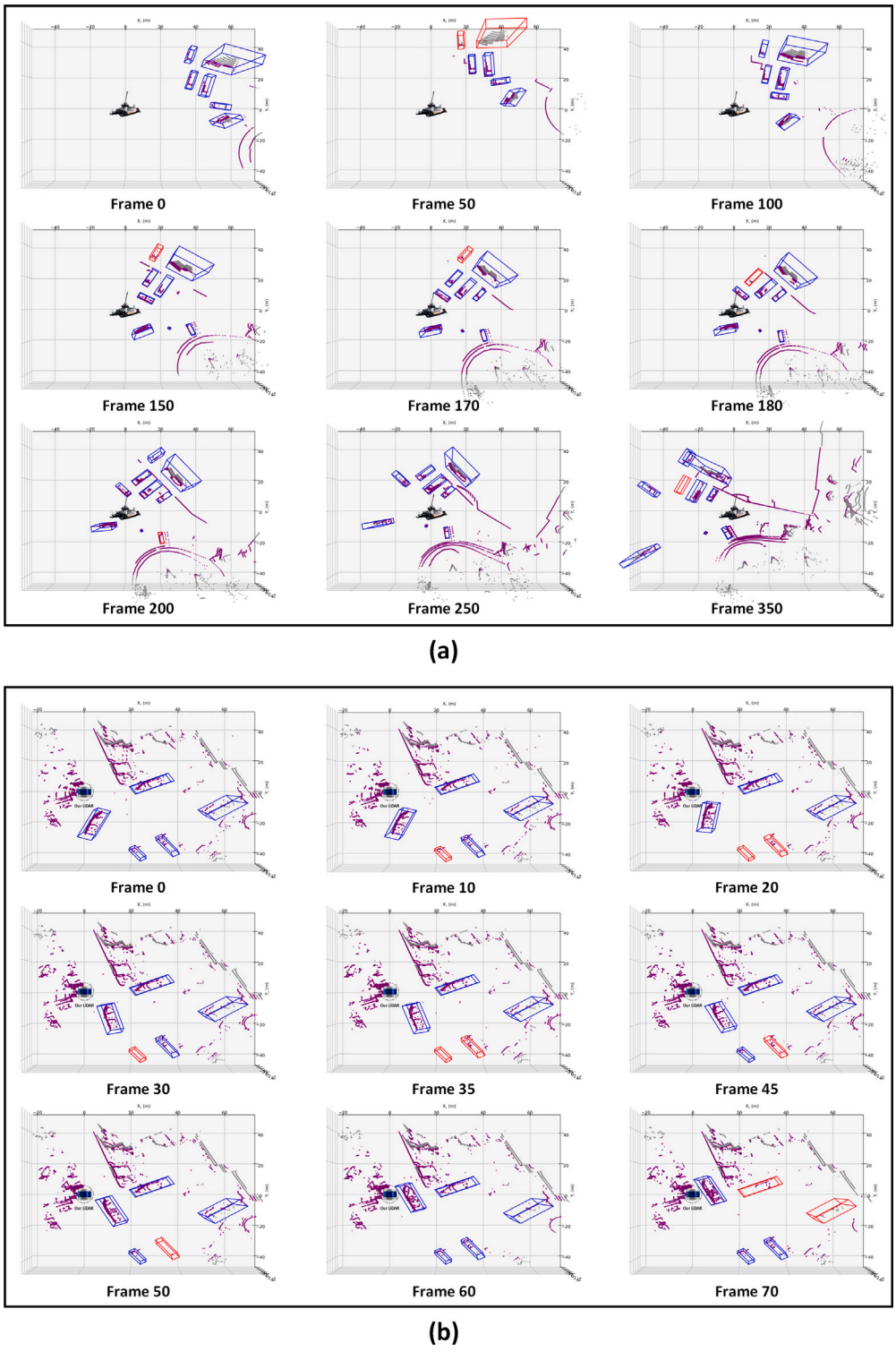


Fig. 15. Visualised tracking results for the (a) simulated scenario and (b) real-world scenario. The blue bounding boxes are the detection results from the detection network, and the red bounding boxes are the compensated results. The purple points indicate the point clouds within the detection range, and the grey point clouds are outside the range.

Our findings demonstrated that while the detection networks trained with simulated datasets performed with a high degree of accuracy when using simulated data, their generalisation to real-world data was initially poor. This highlighted the need for real-world datasets specifically designed for maritime applications. Consequently, we collected and labelled the first-ever real-world LiDAR dataset across the River Thames, which included various types of ships with lengths ranging from 5 m to 40 m. Despite the challenges in labelling maritime point cloud datasets, we developed a strategy to maintain the consistency of target ships across frames. Additionally, the collected real-world datasets were processed and organised similarly to the well-known KITTI datasets to ensure compatibility with most point cloud-based object detection networks.

The simulated datasets were refined and used to train different detection networks. Our results showed that PointPillar demonstrated an overall improvement of 7.2% in detection performance compared

to the results reported in the original paper that introduced these simulated datasets. Furthermore, PV-RCNN, which returned the best detection accuracy, showed an overall improvement of 25.3% compared to the performance reported in the original paper. We also found that voxel-based detection networks outperformed pillar-based detection networks when it came to detecting small objects. Training the networks with real-world data significantly improved their performance in real-world applications. PointPillar, when trained with real-world datasets, showed an improvement of 112% in mAP compared to when trained with simulated data. PV-RCNN achieved the highest mAP at 74.1%.

Our MOT module proved reliable in compensating for missed detections, relying solely on detection results without additional measurements such as AIS. The visualised results from both simulated and real-world scenarios affirmed the effectiveness of our proposed framework.

We also identified the limitations of our proposed framework and datasets. The ship detection networks introduced in this paper are likely experiencing overfitting problems, and incorrect and missed detections occasionally occur which would pose significant safety risks for ASVs. Future research should focus on integrating Bayesian networks and collecting more real-world data from diverse scenarios to mitigate these known limitations.

Overall, our study demonstrates the potential of LiDAR-based ship detection methods for enhancing the safety and operational efficiency of ASVs. The framework and datasets that were developed contribute significantly to the advancement of maritime autonomous technology, paving the way for more robust and reliable ASV operations in complex maritime environments.

CRediT authorship contribution statement

Yongchang Xie: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Cassandra Nanlal:** Writing – review & editing, Supervision. **Yuanchang Liu:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The proposed dataset is available at https://figshare.com/articles/dataset/Maritime_LiDAR-based_Ship_Detection_Datasets/26144536.

Acknowledgements

This work was supported by the Department of Mechanical Engineering (Ye Li and Yanhong Huang) and the Department of Civil, Environmental, and Geomatic Engineering at University College London (UCL), and the Port of London Authority (PLA) (John Dillon-Leetch, Peter Durkin, and Jim Powell). It was also partially funded by the Engineering and Physical Sciences Research Council (EPSRC) under Grant EP/Y000862/1, EP/X034909/1.

References

Bar-Shalom, Y., Li, X.-R., Kirubarajan, T., 2004. Estimation with applications to tracking and navigation: theory algorithms and software, first ed. Wiley, New York.
Bovcon, B., Muhovic, J., Pers, J., Kristan, M., 2019. The MaSTr1325 dataset for training deep USV obstacle detection models. In: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 3431–3438.
Bovcon, B., Muhovic, J., Vranac, D., Mozetic, D., Pers, J., Kristan, M., 2022. MODS-A USV-oriented object detection and obstacle segmentation benchmark. IEEE Trans. Intell. Transp. Syst. 23 (8), 13403–13418.
Caesar, H., Bankiti, V., Lang, A.H., Vora, S., Liong, V.E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G., Beijbom, O., 2020. Nuscenes: A multimodal dataset for autonomous driving. In: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition. pp. 11618–11628.
Cao, X., Gao, S., Chen, L., Wang, Y., 2020. Ship recognition method combined with image segmentation and deep learning feature extraction in video surveillance. Multimedia Tools Appl. 79 (13–14), 9177–9192.
Charles, R.Q., Su, H., Kaichun, M., Guibas, L.J., 2017. PointNet: Deep learning on point sets for 3D classification and segmentation. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition. CVPR, IEEE, pp. 77–85.
Chen, X., Dou, S., Song, T., Wu, H., Sun, Y., Xian, J., 2024a. Spatial-temporal ship pollution distribution exploitation and harbor environmental impact analysis via large-scale AIS data. J. Mar. Sci. Eng. 12 (6), 960.
Chen, X., Wei, C., Yang, Y., Luo, L., Biancardo, S.A., Mei, X., 2024b. Personnel trajectory extraction from port-like videos under varied rainy interferences. IEEE Trans. Intell. Transp. Syst. 25 (7), 6567–6579.
Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., Schiele, B., 2016. The cityscapes dataset for semantic urban scene understanding. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition. CVPR, 2016-, IEEE, pp. 3213–3223.
Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q., 2018. The unmanned aerial vehicle benchmark: Object detection and tracking. In: Computer Vision – ECCV 2018. In: Lecture Notes in Computer Science, vol. 11214, Springer International Publishing, Cham, pp. 375–391.
Geiger, A., Lenz, P., Stiller, C., Urtasun, R., 2013. Vision meets robotics: The KITTI dataset. Int. J. Robotics Res. 32 (11), 1231–1237.
Hansen, K.F., Yao, L., Ren, K., Wang, S., Liu, W., Liu, Y., 2023. Image segmentation in marine environments using convolutional LSTM for temporal context. Appl. Ocean Res. 139, 103709.
Hsieh, M.-R., Lin, Y.-L., Hsu, W.H., 2017. Drone-based object counting by Spatially Regularized Regional proposal network. In: 2017 IEEE International Conference on Computer Vision. ICCV, 2017-, IEEE, pp. 4165–4173.
Hu, X., Zheng, Z., Chen, D., Sun, J., 2023. Autonomous vehicle's impact on traffic: Empirical evidence from waymo open dataset and implications from modelling. IEEE Trans. Intell. Transp. Syst. 24 (6), 1–14.
Huang, X., Wang, P., Cheng, X., Zhou, D., Geng, Q., Yang, R., 2020. The ApolloScape open dataset for autonomous driving and its application. IEEE Trans. Pattern Anal. Mach. Intell. 42 (10), 2702–2719.
Islam, M.S., Han, H., Lee, J.-I., Jung, M.-G., Chong, U., 2013. Small target detection and noise reduction in marine radar systems. IERI Procedia 4, 168–173.
Jiang, X., Cai, J., Wang, B., 2024. YOLOSeaShip: a lightweight model for real-time ship detection. Eur. J. Remote Sens. 57 (1).
Kordic, V., Kordic, V., 2010. Kalman filter / edited by vedran kordic.. Kalman Filter. IntechOpen, Place of publication not identified.
Kuhn, H.W., 2005. The Hungarian method for the assignment problem. Nav. Res. Logist. 52 (1), 7–21.
Lang, A.H., Vora, S., Caesar, H., Zhou, L., Yang, J., Beijbom, O., 2019. PointPillars: Fast Encoders for Object Detection from Point Clouds. arXiv.org.
Lin, J., Diekmann, P., Framing, C.-E., Zweigel, R., Abel, D., 2022. Maritime environment perception based on deep learning. IEEE Trans. Intell. Transp. Syst. 23 (9), 15487–15497.
Lin, T.-Y., Goyal, P., Girshick, R., He, K., Dollar, P., 2020b. Focal loss for dense object detection. IEEE Trans. Pattern Anal. Mach. Intell. 42 (2), 318–327.
Lin, J., Koch, L., Kurowski, M., Gehrt, J.-J., Abel, D., Zweigel, R., 2020a. Environment perception and object tracking for autonomous vehicles in a harbor scenario. In: 2020 IEEE 23rd International Conference on Intelligent Transportation Systems. ITSC, pp. 1–6.
Lin, J., Puthiyavinayagam, A., Liu, S., Kurowski, M., Gehrt, J.-J., Zweigel, R., Abel, D., 2021. Real-time multi-object tracking using adaptive filtering and filter banks for maritime applications. In: 2021 European Control Conference. ECC, pp. 2239–2244.
Liu, Z., Zhang, Y., Yu, X., Yuan, C., 2016. Unmanned surface vehicles: An overview of developments and challenges. Annu. Rev. Control 41, 71–93.
Pitropov, M., Garcia, D.E., Rebello, J., Smart, M., Wang, C., Czarnecki, K., Waslander, S., 2021. Canadian adverse driving conditions dataset. Int. J. Robotics Res. 40 (4–5), 681–690.
Pitropov, M., Huang, C., Abdelzad, V., Czarnecki, K., Waslander, S., 2022. LiDAR-MIMO: Efficient Uncertainty Estimation for LiDAR-based 3D Object Detection. arXiv.org.
Prasad, D.K., Rajan, D., Rachmawati, L., Rajabally, E., Quek, C., 2017. Video processing from electro-optical sensors for object detection and tracking in a maritime environment: A survey. IEEE Trans. Intell. Transp. Syst. 18 (8), 1993–2016.

Qu, J., Liu, R.W., Guo, Y., Lu, Y., Su, J., Li, P., 2023. Improving maritime traffic surveillance in inland waterways using the robust fusion of AIS and visual data. Ocean Eng. 275, 114198.

Ros, G., Sellart, L., Materzynska, J., Vazquez, D., Lopez, A.M., 2016. The SYNTHIA dataset: A large collection of synthetic images for semantic segmentation of urban scenes. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition. CVPR, IEEE, pp. 3234–3243.

Shamsoshoara, A., Afghah, F., Razi, A., Zheng, L., Fulé, P.Z., Blasch, E., 2021. Aerial imagery pile burn detection using deep learning: The flame dataset. Comput. Netw. (Amst. Netherl. : 1999) 193, 108001.

Shan, Y., Yao, X., Lin, H., Zou, X., Huang, K., 2021. Lidar-based Stable Navigable Region detection for unmanned surface vehicles. IEEE Trans. Instrum. Meas. 70, 1–13.

Shi, S., Guo, C., Jiang, L., Wang, Z., Shi, J., Wang, X., Li, H., 2021a. PV-RCNN: Point-Voxel Feature Set Abstraction for 3D Object Detection. arXiv.org.

Shi, S., Wang, X., Li, H., 2019. PointRCNN: 3D Object Proposal Generation and Detection from Point Cloud. arXiv.org.

Shi, S., Wang, Z., Shi, J., Wang, X., Li, H., 2021b. From points to parts: 3D object detection from point cloud with part-aware and part-aggregation network. IEEE Trans. Pattern Anal. Mach. Intell. 43 (8), 2647–2664.

Smith, L.N., 2018. A Disciplined Approach to Neural Network Hyper-Parameters: Part 1 – Learning Rate, Batch Size, Momentum, and Weight Decay. arXiv.org.

Stanislas, L., Dunbabin, M., 2019. Multimodal sensor fusion for robust obstacle detection and classification in the maritime robotx challenge. IEEE J. Ocean. Eng. 44 (2), 343–351.

Sun, Y., Cao, B., Zhu, P., Hu, Q., 2022. Drone-based RGB-infrared cross-modality vehicle detection via uncertainty-aware learning. IEEE Trans. Circuits Syst. Video Technol. 32 (10), 6700–6713.

Varga, L.A., Kiefer, B., Messmer, M., Zell, A., 2022. SeaDronesSee: A maritime benchmark for detecting humans in open water. In: 2022 IEEE/CVF Winter Conference on Applications of Computer Vision. WACV, IEEE, pp. 3686–3696.

Yan, Y., Mao, Y., Li, B., 2018. SECOND: Sparsely embedded convolutional detection. Sens. (Basel, Switz.) 18 (10), 3337.

Yao, L., Kanoulas, D., Ji, Z., Liu, Y., 2021. ShorelineNet: An efficient deep learning approach for shoreline semantic segmentation for unmanned surface vehicles. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 5403–5409.

Yu, F., Chen, H., Wang, X., Xian, W., Chen, Y., Liu, F., Madhavan, V., Darrell, T., 2020. BDD100k: A diverse driving dataset for heterogeneous multitask learning. In: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition. pp. 2633–2642.

Zhang, W., Jiang, F., Yang, C.-F., Wang, Z.-P., Zhao, T.-J., 2021. Research on unmanned surface vehicles environment perception based on the fusion of vision and lidar. IEEE Access 9, 63107–63121.

Zhang, Q., Shan, Y., Zhang, Z., Lin, H., Zhang, Y., Huang, K., 2024. Multisensor fusion-based maritime ship object detection method for autonomous surface vehicles. J. Field Robotics 41 (3), 493–510.

Zhang, G., Yin, J., Deng, P., Sun, Y., Zhou, L., Zhang, K., 2022. Achieving adaptive visual multi-object tracking with unscented Kalman filter. Sens. (Basel, Switz.) 22 (23), 9106.

Zhu, P., Wen, L., Bian, X., Ling, H., Hu, Q., 2018. Vision Meets Drones: A Challenge. arXiv.org.

Zust, L., Kristan, M., 2022. Temporal context for robust maritime obstacle detection. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, 2022-, IEEE, pp. 6340–6346.